

```

import { CITIES_DATA, SALARIES, LICENSES, SECTOR_FINANCIALS, detectSector, getCityBrief, getSectorBrief, getFinancialBrief, getCompetitorsByBudget } from './utils';

export const runtime = 'nodejs';
export const maxDuration = 60;

function numWithCommas(n) {
  const s = String(Math.round(Number(n) || 0));
  return s.replace(/\B(?=(\d{3})+(?!\d))/g, ",");
}

export async function POST(req) {
  try {
    console.log("=== ROUTE VERSION MARKER: HAMOOR-FIX-2026-09-20-V5 (Zai+OpenRoute)");
    const { idea, sector: userSector, city, budget, extras } = await req.json();
    console.log("Request:", { idea, userSector, city, budget, extras });

    if (!idea || !city || !budget) {
      return Response.json({ error: "البيانات ناقصة" }, { status: 400 });
    }

    const apiKey = process.env.GEMINI_API_KEY;
    if (!apiKey) {
      return Response.json({ error: "غير موجود API مفتاح" }, { status: 500 });
    }

    const budgetNum = parseInt(budget);
    const cityName = city.split(" - ")[0].trim();
    const neighborhood = city.includes(" - حي ") ? city.split(" - حي ")[1].trim() : null;

    const sector = userSector ? userSector : detectSector(idea);

    const cityBrief = getCityBrief(cityName) || cityName;
    const sectorBrief = getSectorBrief(sector);
    const financialBrief = getFinancialBrief(sector);
    const suggestedCompetitors = getCompetitorsByBudget(sector, budgetNum);

    // === معلومات إضافية من المستخدم ===
    let extraInfo = "";
    if (extras) {
      const lines = [];
      if (extras.area) lines.push(`مساحة المحل: ${extras.area} متر مربع`);
      if (extras.actual_rent) lines.push(`الإيجار السنوي الفعلي (أكده المستخدم): $${extras.actual_rent}`);
      if (extras.staff_count) lines.push(`عدد الموظفين المتوقع: ${extras.staff_count}`);
      if (extras.shop_state) lines.push(`حالة المحل: ${extras.shop_state}`);
      extraInfo = lines.join("\n");
    }

    return Response.json({
      idea,
      userSector,
      city,
      budget,
      sector,
      cityBrief,
      sectorBrief,
      financialBrief,
      suggestedCompetitors,
      extraInfo
    });
  } catch (error) {
    console.error("Error in POST handler:", error);
    return Response.json({ error: "حدث خطأ غير متوقع" }, { status: 500 });
  }
}

```

```

    if (extras.experience) lines.push(`خبرة صاحب المشروع في هذا المجال`);
    if (lines.length) extraInfo = "\n\n(استخدمها لرفع دقة التحليل)";
}

// === معطيات المشروع المشتركة ===
const projectContext = `المشروع: ${idea}
القطاع: ${sector}
المدينة: ${cityName}${neighborhood ? `الحي: ${neighborhood}` : ''}
الميزانية: ${numWithCommas(budgetNum)} ريال

${cityBrief}

${sectorBrief}

${financialBrief}

بيعات السعودية ${SALARIES.emp_saudi} الرواتب التقريبية (محدثة لـ 2026): موظف سعودي
${LICENSES.commercial_register}: ${LICENSES.municipal_

// === تعليمات الأسلوب ===
const styleGuide = `اشرح "لماذا" وراء كل رقم. لا مجاملة - كن صادقاً وواقعياً`

// === محزك التكيف الذكي حسب نوع المشروع ===
const adaptEngine = `وكيف التحليل
ني، خدمة، رقمي/أونلاين، منتج، فكرة ابتكارية)
ر ولا ديكور؛ استخدم تكاليف تطوير واستضافة وتراخيص رقمية. البنود غير المنطبقة = 0
احسب بالعملاء الشهرين أو عدد العقود، ومتوسط الفاتورة قد يكون قيمة عقد أو اشتراك
.مشروع منزلي/صغير: قلل التكاليف، لا تفرض تكاليف محل كامل
.فكرة ابتكارية: حللها بمنطقها الخاص، اذكر الفرصة والغموض بصراحة
.المنافسون لمشروع غير تقليدي: اذكر البدائل أو الحلول القريبة
. البنود غير المنطبقة = 0 مع شرح في الملاحظات`

// === تعليمات البحث ===
const searchInstructions = `والحي والمعلومات الإضافية التي قدمها صاحب المشروع

لغة: عربية فصحي فقط، ممنوع أي حرف من لغة أخرى. أرقام عادية`

قواعد الدقة:
- ث الحقيقي أولوية مطلقة: إذا توفرت "نتائج البحث الحقيقي من الإنترنت" أسفل هذا الـ
- لا، ثم القائمة المعطاة. ممنوع أسماء وهمية أو عامة. حلل كل منافس بقوة وثغرة محددة
- تكاليف: إذا ظهرت أرقام في البحث، استخدمها. إذا تعارضت مع توقعك، البحث أحدث وأدق
- اذكر صراحة "حسب البحث..." عند الاعتماد على معلومة من البحث recommendations في
- ام واقعية معذلة حسب حجم المدينة والحي. المجاميع تتطابق (مجموع البنود = الإجمالي)
- (2) total = اجمع بنود التأسيس يدوياً وتأكد (1) JSON تحقق رياضي إلزامي قبل إرجاع
- تحت 35 + توضيح كم ينقص score = ميزانية أقل من الحد الأدنى للتأسيس
- ما أعطيت قائمة، اذكر شركات وسلاسل سعودية حقيقية. ممنوع أسماء وهمية أو أوصاف عامة
- ROI = نقطة التعادل = إجمالي التأسيس ÷ (الإيراد عند الاستقرار - التكلفة الشهرية)
- صارم: لو الميزانية ≤ إجمالي التأسيس فهي كافية. لا تقل "غير كافية" إلا لو فعلاً أقل

```

- ل قوة يدعم المشروع، لأنه يعطيه مرونة مالية لمواجهة التحديات والتوسع. ارفع النقاط
- هري الفعلي $\times 12$ - التكاليف السنوية، وعادة يكون متوازناً أو سالباً بسبب فترة النمو
- الربح: لازم يكون منطقياً مع نقطة التعادل. تعادل 10 شهور = ربح سنة أولى بسيط لا ضخم
- اكتساب، نسبة تكرار، إشغال). ممنوع نسب مزيفة مثل "جودة 95%" لمشروع ما بدأ KPIs:
- اسي وأجهزة، مرايا؛ تقني: أجهزة وخواص). مجموعها = قيمة 3-5 equipment_breakdown:
- $\{cityName\}$ الأحياء حقيقية من
- ع لا يحتاج موقعاً فيزيائياً (تطبيق، خدمة أونلاين، متجر إلكتروني، خدمة عن بُعد)، اضبط

```
// == الاستدعاء الأول: التحليل الأساسي ==
const promptCore = `بصرامة وواقعية تامة كدراسة جدوى حقيقية مبنية على بحث فعلي`

${projectContext}

${searchInstructions}

${styleGuide}

${adaptEngine}

${baseRules}

markdo بدون أي نص قبله أو بعده، بدون علامات) فقط JSON بعد بحثك، أرجع النتيجة بصيغة

{
  "score": "<0-100 واقعي>,",
  "decision": "<قرار صريح 6-10 كلمات>,",
  "decision_type": "<positive أو negative>,",
  "summary": "هل ميزانيتك تكفي؟ كم تحتاج فعلياً؟ ما يقلقني؟ ما الذي يحدد نجاحك؟>:",
  "market_demand": "<منخفض/متوسط/عالي/عالي جداً>,",
  "competition": "<منخفضة/متوسطة/عالية/عالية جداً>,",
  "cost_level": "<منخفض/متوسط/عالي/عالي جداً>,",
  "risk_level": "<منخفض/متوسط/عالي/عالي جداً>,",
  "market_analysis": {
    "market_size": "<بأرقام وتقدير قيمته {cityName} حجم السوق لـ>,",
    "target_audience": "<وصف تفصيلي للجمهور: من هم، أعمارهم، دخلهم، سلوكهم>,",
    "buying_patterns": "<أنماط الشراء: متى يشترون، كم مرة، كم ينفقون>,",
    "seasonality": "<الموسمية: أشهر الذروة وأشهر الركود ولماذا>,",
    "expected_market_share": "<النسبة الواقعية مع تبرير>,",
    "growth_potential": "<النمو على 5 سنوات مع تبرير>,",
    "demand_drivers": "[<عامل يرفع الطلب>,<عامل آخر>,<عامل ثالث>]",
    "market_gaps": "[<فجوة في السوق تقدر تستغلها>,<فجوة أخرى>]",
    "competitors": [
      { "name": "<اسم سلسلة حقيقية ظهرت في البحث>","market_position": "ية/متوسطة>"},
      { "name": "<اسم سلسلة حقيقية ثانية>","market_position": "<موقعها>","streng"},
      { "name": "<اسم سلسلة حقيقية ثالثة>","market_position": "<موقعها>","streng"}
    ]
  },
}
```

```

"financial_analysis": {
  "setup_costs": {"rent_deposit": 0, "renovation": 0, "equipment": 0, "licenses": 0},
  "setup_costs_notes": "من سطرين: ما أكبر بند ولماذا، وأين يمكن توفير التكاليف",
  "equipment_breakdown": [
    {"item": "سم بند فرعي من بند المعدات والأثاث، مناسب لنوع هذا المشروع تحديداً", "cost": 0},
    {"item": "<بند فرعي آخر>", "cost": 0},
    {"item": "<بند فرعي آخر>", "cost": 0},
    {"item": "<بند فرعي آخر>", "cost": 0}
  ],
  "monthly_costs": {"rent": 0, "salaries": 0, "utilities": 0, "materials": 0, "other": 0},
  "monthly_costs_notes": "<شرح من سطرين: أثقل بند شهري وكيف تتحكم فيه>",
  "salary_breakdown": [
    {"role": "<مسمى وظيفي>", "count": <عدد>, "monthly_each": <راتب الواحد>},
    {"role": "<مسمى وظيفي>", "count": <عدد>, "monthly_each": <راتب الواحد>}
  ],
  "revenue_projection": {"month_1": 0, "month_3": 0, "month_6": 0, "month_12": 0},
  "revenue_notes": "<شرح من سطرين: كيف ينمو الدخل ولماذا، وما الافتراض الأساسي>",
  "break_even_months": 0,
  "roi_percentage": 0,
  "annual_profit_year1": 0,
  "annual_profit_year3": 0,
  "daily_target": {"customers_per_day": <عدد العملاء المطلوب يومياً للربح>, "average_revenue_per_customer": <متوسط الإيراد لكل عميل>},
  "swot": {
    "strengths": ["<قوة 1>", "<قوة 2>", "<قوة 3>", "<قوة 4>"],
    "weaknesses": ["<ضعف 1>", "<ضعف 2>", "<ضعف 3>"],
    "opportunities": ["<فرصة 1>", "<فرصة 2>", "<فرصة 3>"],
    "threats": ["<تهديد 1>", "<تهديد 2>", "<تهديد 3>"]
  },
  "recommendations": [
    {"title": "<عنوان التوصية>", "detail": "<عملية من سطرين كيف تنفذها ولماذا مهمة>", "priority": "<الأولوية>"},
    {"title": "<عنوان>", "detail": "<شرح عملي>", "priority": "<الأولوية>"},
    {"title": "<عنوان>", "detail": "<شرح عملي>", "priority": "<الأولوية>"},
    {"title": "<عنوان>", "detail": "<شرح عملي>", "priority": "<الأولوية>"},
    {"title": "<عنوان>", "detail": "<شرح عملي>", "priority": "<الأولوية>"}
  ],
  "kpis": [
    {"name": "<مؤشر تشغيلي قابل للقياس فعلياً>", "target": "<قيمة رقمية واقعية>", "description": "<شرح>"},
    {"name": "<مؤشر تشغيلي>", "target": "<قيمة واقعية>", "description": "<شرح>"},
    {"name": "<مؤشر تشغيلي>", "target": "<قيمة واقعية>", "description": "<شرح>"},
    {"name": "<مؤشر تشغيلي>", "target": "<قيمة واقعية>", "description": "<شرح>"}
  ],
  "risk_analysis": [
    {"risk": "<مخاطرة>", "description": "<شرح تفصيلي للمخاطرة وكيف تظهر>", "probability": "<قيمة>"},
    {"risk": "<مخاطرة>", "description": "<شرح تفصيلي>", "probability": "<قيمة>"},
    {"risk": "<مخاطرة>", "description": "<شرح تفصيلي>", "probability": "<قيمة>"},
    {"risk": "<مخاطرة>", "description": "<شرح تفصيلي>", "probability": "<قيمة>"}
  ]
}

```

```

    "risk": "<مخاطرة>", "description": "<شرح تفصيلي>", "probability": "<قيمة>",
  ],
  "is_physical_location": <true (صالون، مطعم، محل، موقع فيزيائي) ...>,
  "locations": {
    "best": {"name": "<حي حقيقي في ${cityName}>", "score": <0-100>, "reason": "<ح
    "worst": {"name": "<حي حقيقي>", "score": <0-100>, "reason": "<شرح>"}
  },
},
}`;

```

// == الاستدعاء الثاني: الخطة والتفاصيل ==

const promptPlan = `على الخطة التنفيذية والتفاصيل العملية، مبنية على بحث فعلي`

\${projectContext}

وع من المشاريع في السعودية وجهات إصدارها، وأسعار السوق للمنتجات والخدمات المشابهة

\${styleGuide}

\${adaptEngine}

قواعد:

- التنفيذية: 3 مراحل واقعية (1-30، 31-60، 61-90 يوم)، كل مرحلة مهام عملية ملموسة -
- ات/خدمات رئيسية، لكل واحد سعر بيع وتكلفة وهامش ربح، أسعار واقعية مبنية على بحثك -
- العميل المثالي: صفه بدقة (العمر، الدخل، السلوك، أين تجده) -
- التراخيص: التصاريح الفعلية الحديثة المطلوبة في السعودية مع جهة الإصدار، من بحثك -
- التميز: 3 طرق عملية ملموسة للتمييز عن المنافسين -
- اكتب بالعربية فقط -

(markdown بدون أي نص قبله أو بعده، بدون علامات) فقط JSON أرجع النتيجة بصيغة

```

{
  "action_plan": [
    {"phase": "30-1 اليوم", "title": "<عنوان المرحلة>", "tasks": ["<مهمة>", "مهمة<
    {"phase": "60-31 اليوم", "title": "<عنوان المرحلة>", "tasks": ["<مهمة>", "م
    {"phase": "90-61 اليوم", "title": "<عنوان المرحلة>", "tasks": ["<مهمة>", "م
  ],
  "pricing": {
    "items": [
      {"name": "<منتج/خدمة>", "price": "<سعر البيع بالريال>", "cost": "<بالريال>
      {"name": "<منتج/خدمة>", "price": "<سعر>", "cost": "<تكلفة>", "margin": "<مش<
      {"name": "<منتج/خدمة>", "price": "<سعر>", "cost": "<تكلفة>", "margin": "<مش<
      {"name": "<منتج/خدمة>", "price": "<سعر>", "cost": "<تكلفة>", "margin": "<مش<
    ],
    "note": "<ملاحظة عن استراتيجية التسعير بأسلوب إنساني>"
  },
  "break_even_detail": {
    "months": "<عدد الأشهر>،

```

```

    "explanation": "<شرح إنساني مبسّط: متى يبدأ المشروع يغطّي تكاليفه ولماذا>",
  },
  "ideal_customer": {
    "age_group": "<الفئة العمرية>",
    "income_level": "<مستوى الدخل>",
    "behavior": "<نمط الحياة والسلوك الشرائي>",
    "where_to_reach": "<أين تجده وكيف توصل له تسويقياً>"
  },
  "licenses_needed": [
    { "name": "<اسم الترخيص>", "issuer": "<جهة الإصدار>" },
    { "name": "<اسم الترخيص>", "issuer": "<جهة الإصدار>" },
    { "name": "<اسم الترخيص>", "issuer": "<جهة الإصدار>" }
  ],
  "differentiation": [
    "<طريقة عملية للتمييز>",
    "<طريقة عملية للتمييز>",
    "<طريقة عملية للتمييز>"
  ]
}
};

```

// === (النموذج الأساسي) Gemini دالة استدعاء ===

```

async function callGemini(userPrompt) {
  // إصدار ثابت - جوجل تحدّثه تلقائياً لأحدث نموذج "flash-latest" alias نستخدم
  // فما نحتاج نرجع نعدّل هذا السطر كل ما يطلعون إصدار جديد
  const model = "gemini-flash-latest";
  const url = `https://generativelanguage.googleapis.com/v1beta/models/${model}`;
  const ctrl = new AbortController();
  const timer = setTimeout(() => ctrl.abort(), 40000);
  let response;
  try {
    response = await fetch(url, {
      method: "POST",
      headers: { "Content-Type": "application/json" },
      body: JSON.stringify({
        contents: [{ role: "user", parts: [{ text: userPrompt }] }],
        // thinkingBudget: 0 الذي يستهلك من نفس حصة 0
        // GEMINI_FAIL_PARSE وهذا كان سبب) قبل اكتماله JSON ويسبب انقطاع الـ
        generationConfig: { temperature: 0.4, maxOutputTokens: 8000, response
      }
    }),
    signal: ctrl.signal
  });
} finally {
  clearTimeout(timer);
}
if (!response.ok) {
  const errText = await response.text();
  console.error("Gemini Error:", response.status, errText.substring(0, 200))
}

```

```

        throw new Error("GEMINI_FAIL_" + response.status);
    }
    const data = await response.json();
    const cand = data.candidates?.[0];
    const text = cand?.content?.parts?.map(p => p.text || "").join("") || "";
    if (!text) throw new Error("GEMINI_FAIL_EMPTY");
    const parsed = extractJSON(text);
    if (!parsed) {
        console.error("Gemini parse failed. finishReason=" + cand?.finishReason +
            throw new Error("GEMINI_FAIL_PARSE");
    }
    return parsed;
}

// == استدعاء Z.ai (GLM) - شركة من شركاء Zhipu
// glm-4.5-flash على حساب إنشاء فقط، تحتاج فقط إنشاء حساب على glm-4.5-flash
async function callZai(userPrompt) {
    const zaiKey = process.env.ZAI_API_KEY;
    if (!zaiKey) throw new Error("ZAI_NO_KEY");
    const ctrl = new AbortController();
    const timer = setTimeout(() => ctrl.abort(), 40000);
    let response;
    try {
        response = await fetch("https://api.z.ai/api/paas/v4/chat/completions", {
            method: "POST",
            headers: { "Content-Type": "application/json", "Authorization": `Bearer`
            body: JSON.stringify({
                model: "glm-4.5-flash",
                messages: [{ role: "user", content: userPrompt }],
                temperature: 0.35,
                max_tokens: 6000,
                response_format: { type: "json_object" }
            }
            },
            signal: ctrl.signal
        });
    } finally {
        clearTimeout(timer);
    }
    if (!response.ok) {
        const errText = await response.text();
        console.error("Z.ai Error:", response.status, errText.substring(0, 200));
        throw new Error("ZAI_FAIL_" + response.status);
    }
    const data = await response.json();
    const text = data.choices?.[0]?.message?.content;
    if (!text) throw new Error("ZAI_FAIL_EMPTY");
    const parsed = extractJSON(text);

```

```

    if (!parsed) throw new Error("ZAI_FAIL_PARSE");
    return parsed;
}

// === عدة نماذج مجانية (صينية وغيرها) بالتتابع - OpenRouter دالة استدعاء ===
// وصول لعشرات النماذج، وبعضها مجاني بالكامل (OPENROUTER_API_KEY) مفتاح واحد
// openrouter.ai/models? من وقت لآخر - راجع OpenRouter القائمة قد تتغير من طرف
// .وحدث الأسماء أدناه لو أحد النماذج تقاعد أو تغير اسمه
const OPENROUTER_FREE_MODELS = [
    "deepseek/deepseek-chat-v3.1:free",
    "z-ai/glm-4.5-air:free",
    "qwen/qwen3-235b-a22b:free",
    "moonshotai/kimi-k2:free",
    "meta-llama/llama-3.3-70b-instruct:free"
];

async function callOpenRouter(userPrompt) {
    const orKey = process.env.OPENROUTER_API_KEY;
    if (!orKey) throw new Error("OPENROUTER_NO_KEY");
    let lastErr = null;
    for (const model of OPENROUTER_FREE_MODELS) {
        const ctrl = new AbortController();
        const timer = setTimeout(() => ctrl.abort(), 40000);
        try {
            let response;
            try {
                response = await fetch("https://openrouter.ai/api/v1/chat/completions", {
                    method: "POST",
                    headers: { "Content-Type": "application/json", "Authorization": `Be` },
                    body: JSON.stringify({
                        model,
                        messages: [{ role: "user", content: userPrompt }],
                        temperature: 0.35,
                        max_tokens: 6000,
                        response_format: { type: "json_object" }
                    })
                });
                signal: ctrl.signal
            }
        } finally {
            clearTimeout(timer);
        }
        if (!response.ok) {
            const errText = await response.text();
            console.error("OpenRouter Error (" + model + "):", response.status, errText);
            lastErr = new Error("OPENROUTER_FAIL_" + response.status);
            continue;
        }
    }
}

```

```

    const data = await response.json();
    const text = data.choices?.[0]?.message?.content;
    if (!text) { lastErr = new Error("OPENROUTER_FAIL_EMPTY"); continue; }
    const parsed = extractJSON(text);
    if (!parsed) { lastErr = new Error("OPENROUTER_FAIL_PARSE"); continue; }
    console.log("OpenRouter succeeded with model: " + model);
    return parsed;
  } catch (e) {
    clearTimeout(timer);
    console.error("OpenRouter exception (" + model + "):", e.message);
    lastErr = e;
    continue;
  }
}
throw lastErr || new Error("OPENROUTER_ALL_FAILED");
}

// === دالة استدعاء Groq (الاحتياطي) ===
async function callGroq(userPrompt, attempt = 1) {
  const groqKey = process.env.GROQ_API_KEY;
  if (!groqKey) throw new Error("لا يوجد مفتاح احتياطي");
  const ctrl = new AbortController();
  const timer = setTimeout(() => ctrl.abort(), 45000);
  let response;
  try {
    response = await fetch("https://api.groq.com/openai/v1/chat/completions",
      method: "POST",
      headers: { "Content-Type": "application/json", "Authorization": `Bearer`
        body: JSON.stringify({
          model: "openai/gpt-oss-120b",
          messages: [{ role: "user", content: userPrompt }],
          temperature: 0.35,
          max_tokens: 3200,
          response_format: { type: "json_object" }
        })
      },
      signal: ctrl.signal
    );
  } finally {
    clearTimeout(timer);
  }
  if (!response.ok) {
    const errText = await response.text();
    console.error("Groq Error:", response.status, errText.substring(0, 300));
    if (response.status === 429 && attempt < 3) {
      await new Promise(r => setTimeout(r, 4000 * attempt));
      return callGroq(userPrompt, attempt + 1);
    }
  }
}

```

```

        throw new Error("GROQ_FAIL_" + response.status);
    }
    const data = await response.json();
    const text = data.choices?.[0]?.message?.content;
    if (!text) throw new Error("GROQ_FAIL_EMPTY");
    const parsed = extractJSON(text);
    if (!parsed) throw new Error("GROQ_FAIL_PARSE");
    return parsed;
}

// === (الأسرع والأقوى - النموذج الأساسي الجديد) Cerebras دالة استدعاء ===
async function callCerebras(userPrompt, attempt = 1) {
    const cerebrasKey = process.env.CEREBRAS_API_KEY;
    if (!cerebrasKey) throw new Error("CEREBRAS_NO_KEY");
    const ctrl = new AbortController();
    const timer = setTimeout(() => ctrl.abort(), 30000);
    let response;
    try {
        response = await fetch("https://api.cerebras.ai/v1/chat/completions", {
            method: "POST",
            headers: { "Content-Type": "application/json", "Authorization": `Bearer`
            body: JSON.stringify({
                model: "gpt-oss-120b",
                messages: [{ role: "user", content: userPrompt }],
                temperature: 0.35,
                max_tokens: 6000,
                response_format: { type: "json_object" }
            }
            signal: ctrl.signal
        });
    } finally {
        clearTimeout(timer);
    }
    if (!response.ok) {
        if (response.status === 429 && attempt < 2) {
            await new Promise(r => setTimeout(r, 3000));
            return callCerebras(userPrompt, attempt + 1);
        }
        const errText = await response.text();
        console.error("Cerebras Error:", response.status, errText.substring(0, 20));
        throw new Error("CEREBRAS_FAIL_" + response.status);
    }
    const data = await response.json();
    const text = data.choices?.[0]?.message?.content;
    if (!text) throw new Error("CEREBRAS_FAIL_EMPTY");
    const parsed = extractJSON(text);
    if (!parsed) throw new Error("CEREBRAS_FAIL_PARSE");

```

```

    return parsed;
}

// == دالة البحث الحي عبر Linkup ==
async function searchLinkup(query) {
    const linkupKey = process.env.LINKUP_API_KEY;
    if (!linkupKey) return null;
    const ctrl = new AbortController();
    const timer = setTimeout(() => ctrl.abort(), 12000);
    try {
        const response = await fetch("https://api.linkup.so/v1/search", {
            method: "POST",
            headers: { "Content-Type": "application/json", "Authorization": `Bearer`
            body: JSON.stringify({
                q: query,
                depth: "deep",
                outputType: "searchResults",
                includeImages: false
            }
            },
            signal: ctrl.signal
        });
        clearTimeout(timer);
        if (!response.ok) {
            console.error("Linkup Error:", response.status);
            return null;
        }
        const data = await response.json();
        const results = data.results || [];
        console.log("Linkup query '" + query.substring(0, 50) + "': " + results.length);
        return results.slice(0, 5).map(r => ({
            title: (r.name || r.title || "").substring(0, 100),
            snippet: (r.content || r.snippet || "").substring(0, 350),
            url: r.url || ""
        }));
    } catch (e) {
        clearTimeout(timer);
        console.error("Linkup exception:", e.message);
        return null;
    }
}

```

```

// == دالة البحث الحي عبر Tavily - بدون بطاقة 1000 Linkup بديل/رديف مجاني لـ Tavily ==
async function searchTavily(query) {
    const tavilyKey = process.env.TAVILY_API_KEY;
    if (!tavilyKey) return null;
    const ctrl = new AbortController();
    const timer = setTimeout(() => ctrl.abort(), 12000);

```

```

try {
  const response = await fetch("https://api.tavily.com/search", {
    method: "POST",
    // لتفادي أي فرق بتوثيق (body + header) نرسل المفتاح بالطريقتين
    headers: { "Content-Type": "application/json", "Authorization": `Bearer`
    body: JSON.stringify({
      api_key: tavilyKey,
      query,
      search_depth: "basic", // 1 الحصة المجانية على
      max_results: 5,
      include_answer: false
    })},
    signal: ctrl.signal
  });
  clearTimeout(timer);
  if (!response.ok) {
    console.error("Tavily Error:", response.status);
    return null;
  }
  const data = await response.json();
  const results = data.results || [];
  console.log("Tavily query '" + query.substring(0, 50) + "': " + results.length);
  return results.slice(0, 5).map(r => ({
    title: (r.title || "").substring(0, 100),
    snippet: (r.content || "").substring(0, 350),
    url: r.url || ""
  }));
} catch (e) {
  clearTimeout(timer);
  console.error("Tavily exception:", e.message);
  return null;
}
}

// وازي، ولا تفشل الاثنتين إلا لو فشل المزودان معاً Tavily و Linkup تدمج نتائج
async function searchCombined(query) {
  const [linkupResults, tavilyResults] = await Promise.all([
    searchLinkup(query),
    searchTavily(query)
  ]);
  const combined = [...(linkupResults || []), ...(tavilyResults || [])];
  if (combined.length === 0) return null;
  // إزالة التكرار حسب الرابط
  const seen = new Set();
  const deduped = combined.filter(r => {
    if (!r.url || seen.has(r.url)) return false;
    seen.add(r.url);
  });
}

```

```

    return true;
  });
  // ترتيب النتائج المدمجة: التي تحتوي أرقاماً مالية أولاً (الأكثر فائدة للتحليل)
  const numScore = (t) => {
    const txt = (t.snippet || "") + (t.title || "");
    let s = 0;
    if (/\\d{2,}/.test(txt)) s += 2;
    if (/\\%|\\$|\\ألف|\\مليون|تكلفة|سعر|إيجار|رأس مال/).test(txt)) s += 3;
    if (/\\(2025|2026)/.test(txt)) s += 1;
    return s;
  };
  deduped.sort((a, b) => numScore(b) - numScore(a));
  return deduped.slice(0, 6);
}

```

```

// === يولّد أسئلة بحث محددة للمشروع AI : المرحلة 1 ===
async function generateResearchQueries(idea, sector, city, budget) {
  const queryGenPrompt = `لل أعمال سعودي خبير. مشروع تجاري معروض عليك للتحليل
  الفكرة: ${idea}
  القطاع: ${sector}
  المدينة: ${city}
  الميزانية: ${budget} دولار

```

على الإنترنت لجمع معلومات حقيقية وحديثة عن هذا المشروع تحديداً. الأسئلة يجب أن تكون محددة جداً (ليس "تكاليف المطعم" بل "تكلفة تأسيس مطعم برجر صغير الرياض 2026") -
 - غطي: التكاليف، المنافسين الحقيقيين بالأسماء، حجم السوق والطلب، التراخيص المطلوبة -
 - تستهدف مصادر سعودية حديثة (2025-2026) -
 - Google مكتوبة كما يبحث الشخص في

فقط بهذا الشكل JSON أرجع:

```

{"queries": ["السؤال الأول", "السؤال الثاني", "السؤال الثالث", "السؤال الرابع"]}

```

```

try {
  const result = await callAI(queryGenPrompt);
  const queries = result.queries || [];
  console.log("Generated " + queries.length + " research queries");
  return queries.slice(0, 4);
} catch (e) {
  console.log("Query generation failed, using fallback queries");
  return [
    `السعودي 2026 ${city} ${idea} تكلفة تأسيس`,
    `السعودية منافسين أسماء حقيقية ${city} ${sector}`,
    `السوق السعودي حجم الطلب 2026 ${idea}`,
    `السعودية الشروط 2026 ${sector} تراخيص`
  ];
}

```

```

// === المرحلة 2: بحث عميق متوازي لكل سؤال ===
async function deepResearch(queries) {
  console.log("Starting deep research on " + queries.length + " queries...");
  const searches = await Promise.all(queries.map(q => searchCombined(q)));
  const validResults = [];
  searches.forEach((results, i) => {
    if (results && results.length > 0) {
      validResults.push({ query: queries[i], results });
    }
  });
  console.log("Deep research: " + validResults.length + "/" + queries.length);
  return validResults;
}

// === المرحلة 3: تجميع نتائج البحث في سياق منظم ===
function buildResearchContext(researchData) {
  if (!researchData || researchData.length === 0) {
    return { context: "", hasData: false };
  }
  let context = "\n\n===== نتائج البحث الحقيقي من الإنترنت =====\n";
  context += "لية تم البحث عنها قبل دقائق. اعتمد عليها كمصدر أساسي في التحليل\n";
  researchData.forEach((item, i) => {
    context += ` \n--- بحث ${i+1}: ${item.query} ---\n`;
    item.results.forEach(r => {
      if (r.snippet) {
        context += ` • ${r.snippet}\n`;
      }
    });
  });
  context += "\n===== \n";
  context += "تعليمات استخدام البحث:\n";
  context += "\nاستخرج الأرقام والأسماء الحقيقية من البحث وادمجها في تحليلك -\n";
  context += "\nالمنافسون: استخدم الأسماء التي ظهرت في البحث -\n";
  context += "\nالتكاليف والأسعار: استخدم الأرقام من البحث إن وُجدت -\n";
  context += "\nإذا تعارضت بياناتك الداخلية مع البحث، البحث أحدث وأدق -\n";
  context += "\nإذا كنت اعتمدت على معلومة من البحث recommendations اذكر في -\n";

  // حد أقصى لطول السياق
  if (context.length > 8000) {
    context = context.substring(0, 8000) + "\n... [النص]";
    console.log("Research context truncated to 8000 chars");
  }
  return { context, hasData: true, dataPoints: researchData.length };
}

// === استدعاء ذكي متعدد المزودين: Cerebras → Groq → Gemini ===

```

```

let cerebrasBroken = false; // ذاكرة فشل خلال نفس الطلب

async function callAI(userPrompt) {
  // 1) Cerebras الطلب نفس فشل سابقاً في نفس الطلب
  if (!cerebrasBroken) {
    try {
      return await callCerebras(userPrompt);
    } catch (e1) {
      console.log("Cerebras failed (" + e1.message + ")");
      if (e1.message.includes("401") || e1.message.includes("402") || e1.mess
        cerebrasBroken = true; // الطلب هذا المحاولة بقية هذا الطلب
        console.log("Cerebras unavailable (no credit/key), skipping for rest
      )
    }
  }
  // 2) OpenRouter لكامل، كوين 235) أولاً - فيها نماذج قوية فعلاً ومجانية بالكامل
  try {
    return await callOpenRouter(userPrompt);
  } catch (e2) {
    console.log("OpenRouter failed (" + e2.message + "), trying Z.ai...");
  }
  // 3) Z.ai (GLM-Flash) - احتياطي لو ازدحمت حصة - OpenRouter أخف وأسرع،
  try {
    return await callZai(userPrompt);
  } catch (eZ) {
    console.log("Z.ai failed (" + eZ.message + "), trying Groq...");
  }
  // 4) Groq
  try {
    return await callGroq(userPrompt);
  } catch (e3) {
    console.log("Groq failed (" + e3.message + "), switching to Gemini...");
  }
  // 5) Gemini كخيار أخير
  return await callGemini(userPrompt);
}

console.log("== بدء التحليل بالبحث الحقيقي ==");

// المرحلة 1: توليد أسئلة بحث ذكية
const researchQueries = await generateResearchQueries(idea, userSector, city,

// المرحلة 2: بحث عميق
const researchData = await deepResearch(researchQueries);

// المرحلة 3: تجميع السياق
const { context: searchContext, hasData, dataPoints } = buildResearchContext(

```

```

if (hasData) {
  console.log("Research-driven analysis: " + dataPoints + " research points +
} else {
  console.log("WARNING: No research data, falling back to AI knowledge");
}

// حقن نتائج البحث الحي في الـ prompts
const enrichedPromptCore = promptCore + searchContext;
const enrichedPromptPlan = promptPlan + searchContext;

const [coreData, planData] = await Promise.all([
  callAI(enrichedPromptCore),
  callAI(enrichedPromptPlan)
]);

let merged = { ...coreData, ...planData };

// === مرحلة التدقيق: مراجعة سريعة للأرقام مقابل البحث ===
if (hasData) {
  try {
    const fa = merged.financial_analysis || {};
    const auditPrompt = `مدقق مالي صارم. راجع هذه الأرقام من دراسة جدوى لمشروع
- إجمالي التأسيس: ${fa.setup_costs?.total || 0} و
- التكاليف الشهرية: ${fa.monthly_costs?.total || 0} و
- إيراد الشهر 12: ${fa.revenue_projection?.month_12 || 0} و
- صافي ربح السنة الأولى: ${fa.annual_profit_year1 || 0} و
- ميزانية صاحب المشروع: ${budgetNum} و

:ونائج بحث حديثة عن السوق
${searchContext.substring(0, 3000)}`;

    ث. إن وجدت رقماً بعيداً بوضوح عن واقع السوق (أعلى أو أدنى بأكثر من 50%)، صخه. أرجع
    {"corrections_needed": <true|false>, "setup_total": <الرقم الصحيح أو نفسه>, "mont
    const audit = await callAI(auditPrompt);
    if (audit && audit.corrections_needed) {
      const fa2 = merged.financial_analysis || {};
      if (audit.setup_total > 0 && fa2.setup_costs) {
        const ratio = audit.setup_total / (fa2.setup_costs.total || audit.set
        if (ratio > 0.3 && ratio < 3) { // حماية من تصحيحات جنونية
          Object.keys(fa2.setup_costs).forEach(k => {
            if (k !== "total" && typeof fa2.setup_costs[k] === "number") {
              fa2.setup_costs[k] = Math.round(fa2.setup_costs[k] * ratio);
            }
          });
        }
      }
    }
    if (audit.month_12_revenue > 0 && fa2.revenue_projection) {

```

```

        const r12 = fa2.revenue_projection.month_12 || 0;
        const ratio = r12 > 0 ? audit.month_12_revenue / r12 : 1;
        if (ratio > 0.3 && ratio < 3) {
            ["month_1", "month_3", "month_6", "month_12"].forEach(k => {
                if (typeof fa2.revenue_projection[k] === "number") {
                    fa2.revenue_projection[k] = Math.round(fa2.revenue_projection[k]
                )
            });
        }
        console.log("Audit pass: corrections applied - " + (audit.audit_note ||
    } else {
        console.log("Audit pass: numbers consistent with market");
    }
    } catch (e) {
        console.log("Audit pass skipped (" + e.message + ")");
    }
}

merged._budget = budgetNum;
merged._sector = sector;
const validated = validateFinancials(merged);
delete validated._budget;
delete validated._sector;

console.log("Analysis complete");
return Response.json(validated);

} catch (error) {
    console.error("Server Error:", error.message);
    // ترجمة الأخطاء التقنية لرسائل واضحة
    let userMsg = "حدث خطأ في التحليل، حاول مرة أخرى";
    const msg = error.message || "";
    if (msg.includes("FAIL_PARSE")) {
        userMsg = "تعذر تحليل المشروع - حاول مرة أخرى بعد لحظات";
    } else if (msg.includes("FAIL_EMPTY")) {
        userMsg = "لم يصل رد كامل من المحلل - حاول مرة أخرى";
    } else if (msg.includes("aborted") || msg.includes("timeout")) {
        userMsg = "التحليل أخذ وقتاً أطول من المتوقع - حاول مرة أخرى";
    } else if (msg.includes("429") || msg.includes("rate")) {
        userMsg = "الخدمة مزدحمة الآن - حاول بعد دقيقة";
    } else if (msg.includes("NO_KEY")) {
        userMsg = "إعدادات الخدمة غير مكتملة - اتصل بالدعم";
    }
    return Response.json({ error: userMsg, debug: msg }, { status: 500 });
}
}

```

// === من نص قد يحتوي على زوائد JSON استخراج ===

```
function extractJSON(text) {
  // محاولة 1: تحليل مباشر
  try { return JSON.parse(text); } catch (e) {}

  // markdown محاولة 2: إزالة علامات
  let cleaned = text.replace(/```json/gi, "").replace(/```/g, "").trim();
  try { return JSON.parse(cleaned); } catch (e) {}

  // محاولة 3: استخراج أول كتلة { ... } متوازنة
  const start = cleaned.indexOf("{");
  if (start === -1) return null;
  let depth = 0, inStr = false, esc = false;
  for (let i = start; i < cleaned.length; i++) {
    const ch = cleaned[i];
    if (esc) { esc = false; continue; }
    if (ch === "\\") { esc = true; continue; }
    if (ch === '"') inStr = !inStr;
    if (inStr) continue;
    if (ch === "{") depth++;
    if (ch === "}") {
      depth--;
      if (depth === 0) {
        const block = cleaned.substring(start, i + 1);
        try { return JSON.parse(block); } catch (e) { return null; }
      }
    }
  }
  return null;
}
```

// === طبقة التحقق الرياضي - تفرض تطابق كل الأرقام ===

```
function validateFinancials(result) {
  try {
    const fa = result.financial_analysis;
    if (!fa) return result;

    // فحص العقلانية مقابل نطاقات القطاع المعروفة (للمشاريع الفيزيائية فقط) 0)
    // المشاريع الرقمية تكاليفها مختلفة جذرياً عن نطاقات القطاعات التقليدية
    const sectorRanges = result._sector ? SECTOR_FINANCIALS[result._sector] : null;
    if (sectorRanges && result.is_physical_location !== false && fa.setup_costs)
      const aiTotal = (fa.setup_costs.rent_deposit||0)+(fa.setup_costs.renovation
      const min = sectorRanges.setup_total?.min || 0;
      const max = sectorRanges.setup_total?.max || Infinity;
      // طلع رقماً أقل من نصف الحد الأدنى أو أكثر من ضعف الأقصى = انحراف صارخ AI لو
      if (aiTotal > 0 && (aiTotal < min * 0.5 || aiTotal > max * 2)) {
```

```

    const target = aiTotal < min * 0.5 ? min : max;
    const ratio = target / aiTotal;
    ["rent_deposit", "renovation", "equipment", "licenses", "initial_inventory",
      if (typeof fa.setup_costs[k] === "number") fa.setup_costs[k] = Math.rou
    });
    result._sanity_note = "لتتوافق مع نطاقات القطاع المعروفة في السوق السعودي";
  }
}

// تصحيح مجموع تكاليف التأسيس (1)
if (fa.setup_costs) {
  const sc = fa.setup_costs;
  sc.total = (sc.rent_deposit||0) + (sc.renovation||0) + (sc.equipment||0) +
    (sc.licenses||0) + (sc.initial_inventory||0) + (sc.marketing_lau
    (sc.working_capital||0);
}

// equipment مطابقة تفصيل المعدات مع بند 1
if (Array.isArray(result.equipment_breakdown) && result.equipment_breakdown.l
  const breakdownSum = result.equipment_breakdown.reduce((s, x) => s + (x.cos
  const equipmentValue = fa.setup_costs.equipment || 0;
  if (breakdownSum > 0 && equipmentValue > 0 && breakdownSum !== equipmentVal
    // equipment نعدل البنود الفرعية بنسبة بحيث يتطابق مجموعها مع بند
    const ratio = equipmentValue / breakdownSum;
    result.equipment_breakdown.forEach(x => { x.cost = Math.round((x.cost ||
  )
}

// تصحيح مجموع التكاليف الشهرية (2)
if (fa.monthly_costs) {
  const mc = fa.monthly_costs;
  mc.total = (mc.rent||0) + (mc.salaries||0) + (mc.utilities||0) +
    (mc.materials||0) + (mc.marketing||0) + (mc.maintenance||0) +
    (mc.other||0);
}

const setupTotal = fa.setup_costs?.total || 0;
const monthlyTotal = fa.monthly_costs?.total || 0;
const rp = fa.revenue_projection || {};
const rev1 = rp.month_1 || 0, rev3 = rp.month_3 || 0;
const rev6 = rp.month_6 || 0, rev12 = rp.month_12 || 0;
const rev24 = rp.year_2_monthly || rev12;
const rev36 = rp.year_3_monthly || rev24;

if (monthlyTotal > 0) {
  // متوسط الإيراد الشهري للسنة الأولى (تدريج واقعي) (3)
  const avgRevY1 = (rev1*2 + rev3*3 + rev6*3 + rev12*4) / 12;

```

```

// 4) صافي الربح السنوي = (متوسط الإيراد - التكاليف) × 12 - رقم واحد محسوب
fa.annual_profit_year1 = Math.round((avgRevY1 - monthlyTotal) * 12);
const avgRevY3 = (rev24 + rev36) / 2;
fa.annual_profit_year3 = Math.round((avgRevY3 - monthlyTotal) * 12);

// 5) نقطة التعادل = التأسيس ÷ صافي الربح الشهري عند الاستقرار - رقم واحد
const stableMonthlyProfit = rev12 - monthlyTotal;
if (stableMonthlyProfit > 0 && setupTotal > 0) {
    let be = Math.ceil(setupTotal / stableMonthlyProfit);
    if (be < 1) be = 1;
    if (be > 120) be = 120;
    fa.break_even_months = be;
    // يمنع التناقض بين الاستدعاءين break_even_detail فرض نفس الرقم على
    if (result.break_even_detail) {
        result.break_even_detail.months = be;
    }
} else {
    // المشروع لا يربح شهرياً = لا توجد نقطة تعادل واقعية
    fa.break_even_months = 0;
    if (result.break_even_detail) result.break_even_detail.months = 0;
}

// 6) ROI = 100 - محسوب × (صافي ربح السنة الأولى ÷ رأس المال)
if (setupTotal > 0) {
    fa.roi_percentage = Math.round((fa.annual_profit_year1 / setupTotal) * 100);

    // راد المستقر، نصح %مبالغ (< 150%) ومتوسط الإيرادات أكبر من 2 ROI ب) لو 6
    // الأولية كانت غير واقعية (مشروع جديد ما يحقق إيراد مستقر من الشهر الأول)
    if (fa.roi_percentage > 150 && rev12 > 0) {
        const expectedAvgRatio = avgRevY1 / rev12;
        // %التدرج الواقعي: المتوسط لازم يكون 55-70% من إيراد الشهر 12، مو 90
        if (expectedAvgRatio > 0.75) {
            // إعادة تدرج الإيرادات بشكل واقعي
            const newRev1 = Math.round(rev12 * 0.25);
            const newRev3 = Math.round(rev12 * 0.45);
            const newRev6 = Math.round(rev12 * 0.70);
            rp.month_1 = newRev1;
            rp.month_3 = newRev3;
            rp.month_6 = newRev6;
            // ROI إعادة حساب المتوسط والربح والـ
            const newAvgY1 = (newRev1*2 + newRev3*3 + newRev6*3 + rev12*4) / 12;
            fa.annual_profit_year1 = Math.round((newAvgY1 - monthlyTotal) * 12);
            fa.roi_percentage = Math.round((fa.annual_profit_year1 / setupTotal) * 100);
        }
    }
}
}
}

```

```

// فحص منطق "الميزانية كافية / غير كافية" - يعتمد على الأرقام فقط (7)
const budget = result._budget || 0;
if (budget > 0 && setupTotal > 0) {
    const enough = budget >= setupTotal;

    // كلمات تدل على أن النص يزعم عدم كفاية الميزانية
    const claimsInsufficient = (txt) => txt && (
        txt.includes("لا تكفي") || txt.includes("غير كافية") ||
        txt.includes("لا تغطي") || txt.includes("أقل من") ||
        txt.includes("إضافية") || txt.includes("عجز") || txt.includes("ينقص")
    );

    if (enough) {
        // الميزانية تكفي فعلياً - نصّح أي زعم بعكس ذلك
        const surplus = budget - setupTotal;
        const wrongDecision = claimsInsufficient(result.decision);
        const wrongSummary = claimsInsufficient(result.summary);

        if (wrongDecision) {
            result.decision = "ميزانيتك كافية لتأسيس المشروع";
            if (result.decision_type === "negative") result.decision_type = "positi
        }

        if (wrongSummary || wrongDecision) {
            // نعيد كتابة بداية الملخص بالأرقام الصحيحة
            result.summary = "ميزانيتك البالغة " + numWithCommas(budget) +
                " ولا تكفي لتغطية تكلفة التأسيس المقدّرة بـ " + numWithCommas(setupTotal)
                " مع فائض قدره " + numWithCommas(surplus) +
                " ولا يمكن استخدامه كرأس مال احتياطي يقوّي وضع مشروعك ";
        }

        // مكافأة الميزانية الفائضة - الفائض الكبير عامل قوة حقيقي يرفع النقاط
        const surplusRatio = surplus / setupTotal; // نسبة الفائض إلى التأسيس
        const currentScore = result.score || 50;
        let scoreBonus = 0;
        if (surplusRatio >= 1.5) scoreBonus = 20; // 150% فائض
        else if (surplusRatio >= 1.0) scoreBonus = 15; // 100% فائض
        else if (surplusRatio >= 0.5) scoreBonus = 10; // 50% فائض
        else if (surplusRatio >= 0.25) scoreBonus = 6; // 25% فائض
        else if (surplusRatio > 0) scoreBonus = 3; // فائض بسيط

        // نطبق المكافأة بحد أقصى 90 (نترك مجال للمشاريع الممتازة)
        const newScore = Math.min(90, currentScore + scoreBonus);
        if (newScore > currentScore) {
            result.score = newScore;
        }
    } else {
        // الميزانية فعلاً لا تكفي - نتأكد أن الملخص يوضّح العجز الصحيح

```

```
const shortage = setupTotal - budget;
result.summary = "ميزانيتك البالغة " + numWithCommas(budget) +
  " ولا تكفي لتغطية تكلفة التأسيس المقدرة بـ " + numWithCommas(setupTotal) +
  " ولا؛ تحتاج إلى " + numWithCommas(shortage) +
  " ولا إضافية. " + (result.summary || "");
}

return result;
} catch (e) {
  console.error("Validation error:", e.message);
  return result;
}
}
```